

TRAINING DECK · 2026

统一开发流程框架

一拖三 Agent · 六阶段 · PDCA 闭环 · TDD 驱动

MASE (Measures AI Software Engineering)

背景与动机

为什么需要统一框架

规范碎片化

OpenSpec 与 Superpower 两套设计文档体系并行，内容重叠，缺乏统一入口

工具无编排

17 个 Skills 散落各处，没有与开发阶段显式绑定，关键环节缺自动化

规则靠手工

TDD、合规检查、代码评审等 8 条规则纯靠人工记忆，无工具化强制执行

目标：一个框架 — 统一规范、编排工具、自动化规则

框架全景

一拖三 Agent · 六阶段



1	需求 Proposal	Plan
2	设计 Design	Plan
3	构建 Build	Do
4	验证 Verify	Check
5	复盘 Retro	Act
6	发布 Release	—

核心理念

管理原则

01

小步快跑

增量开发，每次交付最小可验证单元

02

一拖三制衡

Orchestrator 统筹三 Agent，分工协作、相互制衡

03

复用优先

先查可复用构件清单，杜绝重复造轮子

04

PDCA 闭环

Plan → Do → Check → Act，每一次开发都在进化

工程规则

01 需求澄清确认

原型确认 + 用例对齐后才进入开发

03 TDD 驱动

每个功能必须有自动化测试，先写测试再写代码

05 根因分析

任何改错必须先找到根本原因，不碰运气

07 提交节奏

每 20 次对话执行一次 Git 提交

02 设计方案先行

修改代码之前必做设计方案，杜绝拍脑袋编码

04 合规检查

每次评审必须对照 spec + design 文档

06 系统化解决

杜绝临时补丁，只做系统性根治方案

08 备份机制

删除或回退代码前必须做备份

核心理念

V&V 分层验证

让最擅长验证的那一方去做验证

需求

人 确认

人定标准，AI穷举场景

设计

AI 评审

人只拍板方向

实现

自动化 验证

人不需逐行审阅代码

核心理念

人不能偷懒

V&V 框架只对不偷懒的人有效

01 审阅 GWT 不是点"确认"

逐条想：这个场景会发生吗？边界有价值吗？异常逻辑对吗？

02 拍板方向不是随手选

问AI：为什么推荐这个？另两个坑在哪？选错最坏会怎样？

03 步步回头不是多此一举

设计支撑所有GWT了吗？代码兑现设计了吗？偏了立刻纠正

AI是副驾驶，方向盘在人手里 — 人不握紧，再好的车也到不了

01

Agent 层

一拖三 · 分工协作 · 制衡设计

Orchestrator + 需求 + 开发 + 质量

Agent 1

计划与统管

核心职责

接收需求 → 分解子任务 → 分发 Agent 2/3/4 →
把控门禁 → 最终发布

关键能力

Task 编排 (project-planning-expert) · 进度跟踪 · 门禁检查 · Release

整个框架的中枢 — 不亲自编码，但决定"谁在什么时候做什么"

Agent 2

需求

01 需求探索

brainstorming Skill — 理解目的、约束、成功标准

02 交互原型

frontend-skill — 可交互 HTML 覆盖核心流程

03 Proposal 文档

操作流程 + 系统测试用例表 + 验收标准

产出: proposal.md + HTML原型 + 系统测试用例表

Agent 3

开发

技术预研

AI调研 + 多选一方案对比(优/缺点)
+ POC验证

架构设计

architecture.md + detailed-
design.md + specs/*

TDD 构建

Scenario → 测试 → 实现 → 评审
→ 提交

V&V: AI 自评 + 人拍板方向

AI 做调研给方案对比 → 人看利弊拍板 → AI 完成详细设计 → 回查 GWT 自评一致性

人把握大方向，AI负责调研+细节 — 不是人一个人想，也不是AI一个人定

Agent 4

质量

设计评审

code-quality-controller — 架构合规、需求一致性、
技术可行性

前端评审

frontend-design — 视觉、交互、响应式、无障碍、
性能

代码质量

code-review (含 spec 合规+代码规约) + 安全扫描

BUG 修复

bug-fixer v3 — 根因→系统修复→测试复盘→横向扫描

02

阶段层

六阶段 · 层层递进 · PDCA 闭环

Proposal → Design → Build → Verify → Retro → Release

阶段 1

需求 *Proposal*

A 需求探索

brainstorming — 理解目的、约束、成功标准

B 交互原型

frontend-skill — 可交互 HTML 覆盖全部核心操作流程

C GWT 测试用例

Given-When-Then 穷举 Happy Path + 边界 + 异常

V&V: 人审阅 GWT 用例确认

人不读需求文档，人审阅测试用例 — GWT 精确、无歧义、可传递

产出: proposal.md + HTML原型 + GWT测试用例 | 门禁: 人确认

阶段 2 · Layer 1

技术预研

三张汇总表

技术难点方案、非功能需求满足、外部依赖评估

联网调研

成熟方案借鉴 + 最新技术接口 + 方案对比

POC 验证

所有外部依赖编写验证脚本并跑通

可复用构件清单

提取现有轮子，避免重复造轮子

产出: tech-feasibility.md | 工具: WebSearch + RunCommand (POC)

架构设计

A	architecture.md	架构图 · 技术栈 · 模块边界
B	detailed-design.md	管道流程 · 数据模型 · API · 真实数据快照
C	tasks.md	project-planning-expert
D	specs/{capability}/	GWT 验收规格（来自阶段一）
E	AI 自评	回查GWT + 接口合约一致性 + 边界降级策略

V&V: AI 多轮自评 + 人拍板方向

设计评审交给AI — 不会累、不会漏、不会"差不多就行"

构建 *Build*

1

复用检查

对照可复用构件清单

2

写测试

pytest 单元测试 (来自阶段一的 GWT 用例)

3

写实现 → 跑测试

TDD 内层: 红 → 绿 → 重构

4

功能测试 + 多格式一致性

webapp-testing + HTML/Word/Text 关键词对比

5

代码评审 + 安全扫描

code-review + TRAE-security-review

V&V: 自动化测试 100% 通过

阶段 4

验证 *Verify*

BUG 修复闭环

根因分析 → 系统修复 → 回归验证 → 测试反哺 → 横向
扫描 → 知识沉淀

原则：不找到根因不放过 · 不系统修复不放过 · 不沉淀不放过

工具: bug-fixer v3 (含 TRAE-debugger 运行时调试)

阶段 5

复盘 *Retro*

A

收集

本轮 lessons / cases / 评审记录

B

分析

高发 BUG 类型 · 测试盲区 · 设计缺陷 · 流程短板

C

改进

更新 Skill 规则 · 完善模板 · 强化门禁 · 更新代码规约

每一次开发都在自我进化，而非简单"做完就完"

发布 *Release*

1 最终合规审查

对照全部 spec + design 全量检查

2 使用手册

自动生成 docs/user-guide.md

3 Git Commit

git-commit Skill — Conventional Commit + 变更摘要

4 归档

归档 OpenSpec Change，关闭开发周期

核心理念

PDCA 闭环



Plan 计划

阶段 1+2 — 需求 + 设计



Do 执行

阶段 3 — TDD 微循环构建



Check 检查

阶段 4 — 验证 + BUG 修复



Act 改进

阶段 5 — 复盘 + 系统进化

核心理念

步步回头

每一步往前走之前，回头看一眼上一步的承诺

需求 GWT

整个流水线的唯一真相源

设计文档

接口合约一致吗？边界有降级吗？

代码实现

测试来自GWT，绿灯即正确

需求 GWT ——→ 设计文档 ——→ 代码实现 | 每一步回头对照，偏了立刻纠正

目录结构

src/	产品代码	按 capability 分模块 · 子目录分层
tests/	测试代码	镜像 src/ · unit / integration / fixtures
docs/	文档	user-guide · lessons · cases
openspec/	规范文档	proposal · design · specs · tasks

规则: 每类文件唯一位置 | 镜像原则 | Capability 对齐 | 根目录整洁

工具链

Skills 矩阵

需求阶段

brainstorming
frontend-skill

设计阶段

code-quality-
controller
frontend-design
project-planning-
expert

构建阶段

test-driven-
development
webapp-testing
code-review(含合
规)
TRAE-security-
review

验证阶段

bug-fixer v3
(含 TRAE-
debugger
运行时调试)

发布阶段

git-commit
code-review
(最终合规)

需求阶段 · Skill

brainstorming

探索用户意图、澄清需求边界、对齐成功标准

意图探索

理解用户的真实目的与核心诉求，而非表面功能

约束识别

技术栈、时间、资源、合规等限制条件梳理

可选方案

多方案对比，引导用户做出知情决策

成功标准

可验证的验收条件，为后续 Gherkin 打基础

需求阶段 · Skill

frontend-skill

生成可交互HTML原型，让需求"看得见、点得动"

快速原型

覆盖核心操作流程的可交互HTML页面

视觉确认

可视化确认需求，避免文字描述歧义

操作流程

端到端操作路径复现，验证业务逻辑

快速迭代

基于反馈即时修改原型，低沟通成本

project-planning-expert

将模糊需求转化为结构化、可执行的任务分解

任务分解

按 Capability 拆分子任务，粒度可控

依赖排序

识别任务间依赖，确定执行先后顺序

并行识别

标记可并行任务，支持多Agent协作

产出 tasks.md

结构化任务清单，供 Agent 1 编排调度

code-quality-controller

设计评审：SOLID原则 · GRASP模式 · 架构合规

需求一致性

对照 proposal 检查设计是否完整覆盖需求

设计质量

SOLID / GRASP 原则评审，识别代码坏味道

测试覆盖

检查 spec 覆盖率、边界条件、异常路径

可复用提取

识别可抽取构件，更新复用构件库

设计阶段 · Skill

frontend-design

前端设计评审：视觉规范 · 交互逻辑 · 性能指标

视觉合规

色彩、字体、间距、响应式是否符合设计系统

交互审查

操作流程是否顺畅，异常状态是否覆盖

无障碍

a11y 检查：语义化、焦点、色彩对比度

性能评估

加载速度、资源体积、渲染性能

test-driven-development

TDD 微循环：先写测试 → 再写实现 → 重构优化

外层契约

基于 Gherkin spec 编写 pytest 测试骨架

红 → 绿

先写失败测试 → 最小实现使其通过

重构

消除重复、优化结构，保持测试全绿

最小单元

每次只实现一个 Scenario，小步快跑

webapp-testing

Playwright 驱动的 Web 应用功能测试与调试

浏览器自动化

Playwright 操控真实浏览器执行操作

功能验证

点击、输入、跳转、断言页面状态

截图对比

捕获页面截图，辅助 UI 回归检测

日志调试

查看浏览器控制台日志，快速定位问题

构建阶段 · Skill

code-review

多轮多代理并行评审 · spec合规 · 代码规约检查

Spec 合规

对照 spec 逐条检查实现是否完整覆盖

代码规约

命名、结构、错误处理、类型安全

并行评审

多代理并发检查不同维度，提升效率

结构化反馈

按严重程度分级输出，支持门禁判断

TRAE-security-review

代码安全漏洞扫描与安全最佳实践审查

漏洞扫描

注入攻击、XSS、CSRF、敏感信息泄露

依赖审计

第三方库已知漏洞检查

权限模型

认证、授权、会话管理的安全性

安全规约

加密、日志脱敏、输入校验等最佳实践

验证阶段 · Skill

bug-fixer v3

八大原则驱动的BUG修复闭环 · 含运行时调试

01

根因分析

再动手修复

02

系统化方案

杜绝临时补丁

03

回归验证

通过才算修好

04

测试反哺

补测试+回写GWT

05

横向扫描

一并修复

06

知识沉淀

避免重犯

07

三不放过

· 未沉淀不放过

08

运行时调试

科学调试流程

scientific debugging 流程

假设 → 插桩 → 复现 → 分析 → 修复 → 回归验证 → 沉淀

发布阶段 · Skill

git-commit

Conventional Commit · 智能暂存 · 消息自动生成

类型自动推断

根据 diff 分析 feat/fix/refactor 等类型

范围识别

从变更路径自动提取 scope

消息生成

Conventional Commit 格式的提交信息

智能分组

按逻辑分组暂存，避免大杂烩提交

快速上手

三步启动

1

复制模板

```
cp -r openspec/changes/_template/ openspec/changes/{project}/
```

2

创建目录

按 STRUCTURE.md 创建 src/ tests/ docs/ 分组目录

3

发起需求

对 AI 说: "创建新项目 {name}" — 框架自动运转

命令: "创建新项目"

命令: "复盘总结"

谢谢

每一次开发都是一次进化

PDCA 不是终点，是起点